

<u>CL-2006</u> <u>Operating System Lab</u>	<u>LAB -07</u> <u>Threads and Multithreading</u>
---	---

National University of Computer and Emerging Science
Spring 2026

Table of Content

Objective.....	3
Threads.....	3
Can we write Multithreading Programs in C?.....	3
Thread Library:.....	4
Basic System Calls related to Multithread Programming.....	4
Pthread_create() - System Call.....	4
Pthread_join() - System Call.....	5
Example 1: Two Threads displaying two strings “Hello” and “How are you?” independent of each other.....	6
Example 2: Create a function message() that takes threaded as argument and prints the message with thread id. There should be at least five independent threads.....	7
Attributes in Threads.....	8
System Calls related to Attributes in Threads.....	8
pthread_attr_init() - System Call.....	8
pthread_attr_setdetachstate() - System Call.....	8
pthread_attr_destroy() - System Call.....	9
Example 3: Create a detached thread for a function infoThread().....	9
Lab Activity.....	11

Objective

The objective of this lab is to make you understand and learn about implementing threads using the Pthread library already. At the end of the lab, you will be able to learn:

- Thread creation in Linux
- Joining of thread in Linux
- Initializing thread attributes
- Setting Attribute detach state
- Destroying attribute

Threads

Threads are often described as lightweight processes. They can work like two or more processes sharing the same address space i.e., they will work independently like processes but can share the same global variables. They are mostly used when two tasks can be done independently without depending much on each other.

1. **Provides:** Concurrency and parallelism: Threads allow multiple streams of execution within a single process. This means that different parts of a program can execute independently, potentially concurrently, improving the overall efficiency of the program by utilizing available CPU resources effectively. In certain applications, such as web servers or database servers, threads can be used to handle multiple client requests concurrently, improving scalability and throughput.
2. **Benefits:** Responsiveness, Modularity, Resource sharing, Efficiency: Threads can be used to keep an application responsive to user input or external events while performing other tasks in the background. For example, a user interface can remain interactive even while performing complex computations in separate threads. Threads can be used to break down a complex task into smaller, more manageable units of execution, improving modularity and facilitating easier maintenance and development of software systems. Threads within the same process share resources such as memory and file descriptors, which can lead to efficient communication and data sharing between different parts of a program. While threads introduce overhead due to context switching and synchronization, they can still be more efficient than separate processes in many cases, particularly when sharing data and resources.
3. **Requires:** Synchronization: Threads often need to coordinate their actions to ensure data consistency and avoid race conditions. Synchronization mechanisms such as mutexes, semaphores, and condition variables are used to control access to shared resources and coordinate the execution of threads.

Can we write Multithreading Programs in C?

Unlike Java, multithreading is not supported by the language standard. POSIX Threads (or Pthread) are POSIX standard for threads. Implementation of Pthread is available with the gcc compiler.

Thread Library:

- POSIX Pthread
- Two general strategies for creating multiple threads.
 - Asynchronous threading:
 1. Parent and child threads run independently of each other.
 2. Typically, little data sharing between threads
 - Synchronous threading:
 1. The parent thread waits for all of its children to terminate.
 2. Children threads run concurrently.
 3. Significant data sharing.
- Each thread has a set of attributes, including stack size and scheduling information.
- In a Pthread program, separate threads begin execution in a specified function.
//runner()
- When a program begins
 1. A single thread of control begins in main ()
 2. main() creates a second thread that begins control in the runner() function.
 3. Both threads share the global data

Basic System Calls related to Multithread Programming

The following are two basic system calls related to multithreaded programming; however, there are many system calls available.

S. NO	System Call	Description
1	Pthread_create()	For creating thread
2	Pthread_join()	Wait of thread termination

Pthread_create() - System Call

This system call is used to create a new thread, a syntax is given below.

```
#include <pthread.h>
int pthread_create(
pthread_t *threaded,    //id of thread
const pthread_attr_t *attr, //attributes of thread
void *(*start_routine) (void*), //function that is to assign
void *arg                //arguments that have to pass to thread function
);
```

```
Example: pthread_create(&id[0], NULL, printNumber, &arg);
```

Arguments	Syntax	Description
ID	pthread_t *	Reference (or pointer) to the ID of the thread.
attr	pthread_attr_t *	Used to set the attributes of a thread (e.g., the stack size, scheduling policy, etc.) Passing NULL suffices for most applications.
Starting Routine	void *	The name of the function that the thread starts to execute. If the function's return type is void *, then its name is simply written; otherwise, it must be type-cast to void *.
arg	void *	This is the argument that the starting routine takes. If it takes multiple arguments, a Struct is used.

Return Values:

If successful it returns 0 otherwise it generates a nonzero number.

Pthread_join() - System Call

This system call waits for the thread specified by thread to terminate. A syntax is shown below:

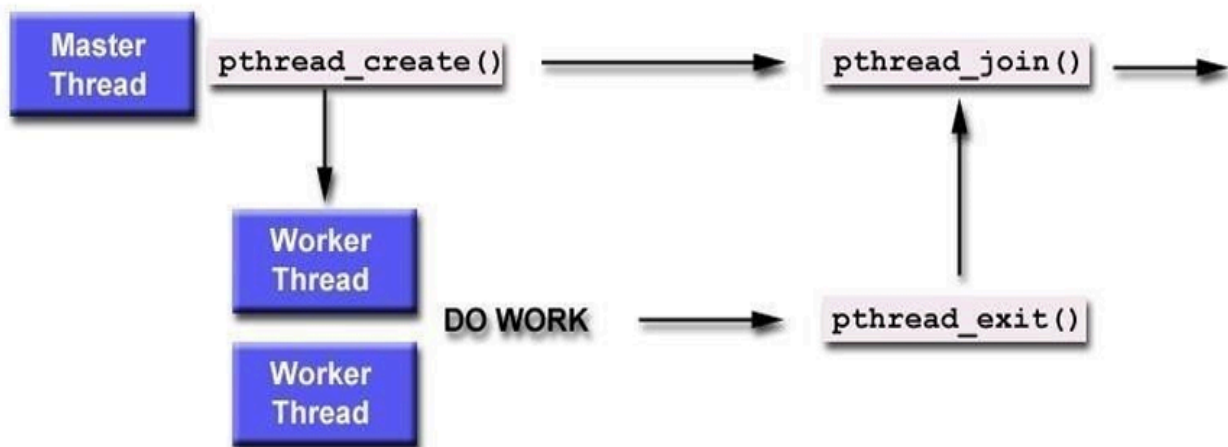
```

Int pthread_join (
    Pthread_t threaded,    //id of thread which have to join
    void **retval          //return status of thread
);

```

Return Values:

If successful it returns 0 otherwise it generates a nonzero number.



Example 1: Two Threads displaying two strings “Hello” and “How are you?” independent of each other

1. Create a new file thread.c with .c extension using any editor.
2. Type the following code.

```
#include <pthread.h>
#include <stdlib.h>
void * thread1(){
while(1){
    printf("Hello!!\n");
}
}
void * thread2(){
while(1){
    printf("How are you?\n");
}
}
int main(){
int status;
pthread_t tid1,tid2;
pthread_create(&tid1,NULL,thread1,NULL);
pthread_create(&tid2,NULL,thread2,NULL);
pthread_join(tid1,NULL);
pthread_join(tid2,NULL);
return 0;
}
```

3. Save and exit.
4. To compile it type the following command on terminal.

```
gcc thread.c -o thread -lpthread
```

Here, **-lpthread** is an option to execute the pthread.h library file.

5. Run it by using the following command.

```
./thread
```

Example 2: Create a function message() that takes thread id as argument and prints the message with thread id. There should be at least five independent threads

1. Create a new file msgthreads.c with .c extension using any editor.
2. Type the following code.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_THREADS 5

void *PrintHello(void *threadid){
    long tid;
    tid = (long)threadid;
    printf("Hello World! It's me, thread #%ld!\n", tid);
    pthread_exit(NULL);
}

int main (int argc, char *argv[]){
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0; t<NUM_THREADS; t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n",
                rc);
            exit(-1);
        }
    }
    pthread_exit(NULL);
}
```

3. Save and exit.
4. To compile it type the following command on terminal.

```
gcc msgthreads.c -o msgthreads -lpthread
```

5. Run it by using the following command.

./msgthreads

Note:

remove pthread_join system call and then observe the changes

Attributes in Threads

Thread attributes are thread characteristics that affect the behavior of the thread. NULL is passed in above examples in place of thread attributes, now we start to place thread attributes that use default attributes of threads. We may create and customize a thread attribute object to specify other values for the attribute.

System Calls related to Attributes in Threads

The following are the system calls related to threads' attributes.

S. NO	System Call	Description
1	pthread_attr_init()	Initializes a thread attributes object
2	pthread_attr_setdetachstate()	Controls detach state of a thread
3	pthread_attr_destroy()	Destroys attribute objects

pthread_attr_init() - System Call

This initializes a thread attributes object attr with the default value. The syntax is shown below:

```
int pthread_attr_init(pthread_attr_t *attr)
```

Return Values:

If successful completion, it will return a 0 otherwise, an error number is returned to indicate the error.

pthread_attr_setdetachstate() - System Call

The detachstate attribute controls whether the thread is created in a detached state.

```
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate)
```

- **PTHREAD_CREATE_DETACHED**

Thread state is detached means it cannot be joined with other threads.

- **PTHREAD_CREATE_JOINABLE**

Thread state is joinable means it can be joined with other threads

pthread_attr_destroy() - System Call

When a thread attributes objects is no longer required, it should be destroyed using this SC.

```
int pthread_attr_destroy(pthread_attr_t *attr)
```

Return Values:

If successful completion, it will return a 0 otherwise, an error number is returned to indicate the error.

[pthread_self\(\)](#)

Syntax:

```
pthread_t tid;  
tid = pthread_self();
```

DESCRIPTION:

The pthread_self() function shall return the thread ID of the calling thread.

RETURN VALUE:

Refer to the description.

ERRORS:

No errors are defined. The pthread_self() function shall not return an error code.

Example 3: Create a detached thread for a function infoThread()

1. Create a new file msgthreads.c with .c extension using any editor.
2. Type the following code.

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void *theThread(void *parm) {
    printf("Entered the thread\n");
    return NULL;
}
int main(int argc, char **argv) {
    pthread_attr_t attr;
    pthread_t thread;
    printf("Create a default thread attributes object\n");
    pthread_attr_init(&attr);
    printf("Set the detach state thread attribute\n");
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    printf("Create a thread using the new attributes\n");
    pthread_create(&thread, &attr, theThread, NULL);
    printf("Destroy thread attributes object\n");
    pthread_attr_destroy(&attr);
    int rc;
    rc = pthread_join(thread, NULL);
    printf("Join now fails because the detach state attribute was
        changed\n pthread_join returns non zero value %d", rc);
    printf("Main completed\n");
    return 0;
}

```

3. Save and exit.
4. To compile it type the following command on terminal.

```
gcc detachthread.c -o detachthread -lpthread
```

5. Run it by using the following command.

```
./detachthread
```

You can get much information about these attributes and more information about system calls related to thread attributes: follow the links below

6. <https://docs.oracle.com/cd/E19455-01/806-5257/6je9h032j/index.html>

7. <http://www.cs.cmu.edu/afs/cs/academic/class/15492-f07/www/pthreads.html>
8. <https://vcansimplify.wordpress.com/2013/03/08/pthread-tutorial-simplified/>

Lab Activity

1. Write a program that creates 3 threads.
 - a) On successful creation, print "Thread #" in its starting routine and terminate themselves by showing their return value.
 - b) On unsuccessful creation, Print "Error".

```
Thread 1
Thread 2
Thread 3
Thread 1 returns: 0
Thread 2 returns: 0
Thread 3 returns: 0
```

2. Write a program, which makes 4 threads. Each thread will print one table out of [5678] up to 1000.
3. Write a program that creates N number of threads specified in the command line, each prints "hello message and its own thread ID". Sleep the main thread for 1 second and create every 4 or 5 threads. The output of your code should alike:

```
I am thread 1. Created new thread (4) in iteration 0...
Hello from thread 4 - I was created in iteration 0
I am thread 1. Created new thread (6) in iteration 1...
I am thread 1. Created new thread (7) in iteration 2...
I am thread 1. Created new thread (8) in iteration 3...
I am thread 1. Created new thread (9) in iteration 4...
I am thread 1. Created new thread (10) in iteration 5...
Hello from thread 6 - I was created in iteration 1
Hello from thread 7 - I was created in iteration 2
Hello from thread 8 - I was created in iteration 3
Hello from thread 9 - I was created in iteration 4
Hello from thread 10 - I was created in iteration 5
I am thread 1. Created new thread (11) in iteration 6...
I am thread 1. Created new thread (12) in iteration 7...
Hello from thread 11 - I was created in iteration 6
Hello from thread 12 - I was created in iteration 7
```

4. Write a program to sum 10 elements of an array by multithreading.
5. Procom has 4 volunteers on their front desk.
 - Volunteer 1 manages On day registration
 - Volunteer 2 handles announcements
 - Volunteer 3 handles sponsors
 - Volunteer 4 resolve queries of participants

Implement this system using pthread for 100 participants.